

Pobierając daną treść (np. z internetu) możemy wybrać format, w jakim dostaniemy informacje do późniejszego przetwarzania:

- TXT / CSV / TSV (praca domowa: sprawdzić czym jest TSV) `text/plain` bądź `csv`
- JSON / YAML `Content-Type: application/json`
- XML
- Excel
- HTML `text/html`
- PDF `Content-Type: application/pdf`

**Serializacja**: proces konwersji obiektu do formatu umożliwiającego jego przechowywanie lub przesyłanie oraz późniejsze odtworzenie.

JSON jest standardem serializacji danych.

// JSON to de facto standard serializacji, gdy chodzi o przesyłanie danych przez HTTP, *lingua franca* używana do wymiany danych między różnorodnymi programami, zarówno w nowoczesnych stronach internetowych, jak i aplikacjach mobilnych.

|                  |                                                                  |
|------------------|------------------------------------------------------------------|
| object           | string                                                           |
| { }              | " "                                                              |
| { members }      | " chars "                                                        |
| members          | chars                                                            |
| pair             | char                                                             |
| pair , members   | char chars                                                       |
| pair             | char                                                             |
| string : value   | any-Unicode-character-<br>except-"-or-\-or-<br>control-character |
| array            | \ " \ f                                                          |
| [ ]              | \\ \n                                                            |
| [ elements ]     | \ / \ r                                                          |
| elements         | \ b \ t                                                          |
| value            | \\u four-hex-digits                                              |
| value , elements | number                                                           |
| value            | int                                                              |
| string           | int frac                                                         |
| number           | int exp                                                          |
| object           | int frac exp                                                     |
| array            | int                                                              |
| true             | digit                                                            |
| false            | digit1-9 digits                                                  |
| null             | - digit                                                          |
|                  | - digit1-9 digits                                                |
|                  | frac                                                             |
|                  | - digits                                                         |
|                  | exp                                                              |
|                  | e digits                                                         |
|                  | digits                                                           |
|                  | digit                                                            |
|                  | digit digits                                                     |
|                  | e                                                                |
|                  | E                                                                |
|                  | e+                                                               |
|                  | E+                                                               |
|                  | e-                                                               |
|                  | E-                                                               |

## Gramatyka JSON.

<https://danepubliczne.imgw.pl/pl/apiinfo>

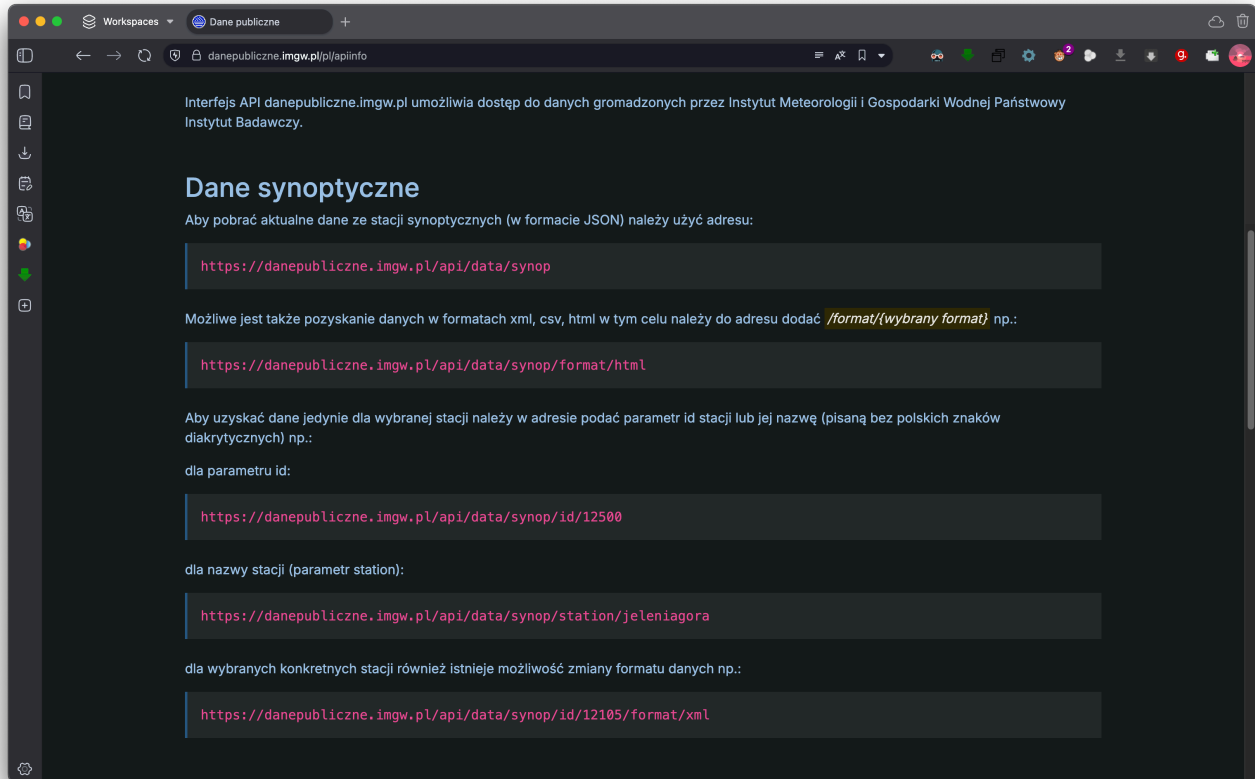
Za pomocą powyższego API domyślnie możemy pobrać interesujące nas dane w formacie JSON:

<https://danepubliczne.imgw.pl/pl/apiinfo>

Istnieje jednak możliwość wskazania innego formatu:

- xml
- csv
- html

<https://danepubliczne.imgw.pl/api/data/synop/format/html>



Serwis danych publicznych umożliwiając pobranie historycznych danych synoptycznych.

Gdy wejdziemy na powyższe linki bezpośrednio w przeglądarce, silnik przeglądarki (V8 - JavaScript) automatycznie wyśle zapytanie GET za pomocą protokołu HTTP (służącego do wymiany informacji w internecie) do usługodawcy. Następnie serwer zwróci pożądane dane, które możemy wyświetlić bezpośrednio w oknie przeglądarki.

## Różnice pomiędzy strukturą danych:

### Ramka danych

- CSV
  - Często stosowane w uczeniu maszynowym
- Excel
  - Często stosowane w Data Science

### Struktura drzewiasta

- JSON

- Często stosowane w komunikacji serwisów internetowych

JSON

```
{
  "owners": [
    {
      "firstName": "Jan",
      "lastName": "Kowalski",
      "age": 42,
      "cars": [
        {
          "brand": "Toyota",
          "model": "Corolla",
          "registrationNumber": "WX12345"
        },
        {
          "brand": "Ford",
          "model": "Focus",
          "registrationNumber": "WE54321"
        }
      ]
    },
    {
      "firstName": "Anna",
      "lastName": "Nowak",
      "age": 35,
      "cars": [
        {
          "brand": "Volkswagen",
          "model": "Golf",
          "registrationNumber": "KR98765"
        }
      ]
    }
  ]
}
```

- YAML
  - Często stosowane, jako dane konfiguracyjne aplikacji

»

## Format tekstowy TXT:

- powstał na potrzeby komunikacji dalekopisów, które musiały interpretować znaki w tych samych kodowaniach pomiędzy różnymi producentami.

Stąd na znak nowej linii składają się dwa znaki:

Carriage Return (CR) - cofnięcie głowicy drukującej

Line Feed (LF) - wysunięcie papieru

Natomiast systemy Unixowe nie zachowały powyższej konwencji i znak nowej linii interpretują tylko jednym kodem, tj. Line Feed.

»

## CVS tj. Comma Separated Values

- format do wymiany ustrukturyzowanych informacji.

Powyższy format, tak naprawdę "nie istnieje" tj. nie ma własnej specyfikacji.

Systemowa funkcja wypisywania liczb np. w terminalu najpierw sprawdzi pole mówiące, jaki separator liczb wydrukować:



Wówczas zapis, takich liczb, oddzielonych przecinkiem, mógłby być problemem:

10,50,7,9,45 itp.

- 10,50
- 7
- 9,45

Jak widać w powyższym przykładzie ciężko zinterpretować liczbę całkowitą w ciągu znaków.

Wstawienie enteru powoduje ustawienie nowego wiersza danych:

- zamiana enteru na znak specjalny tj. `\n`
- `//n` - dwa ukośniki powodują unieważnienie znaku specjalnego.

Interpretacja cudzysłówów `"tekst, "`:

- dwie osobne wartości tj.
  - `"tekst"`
  - `"`
- czy jedna wartość tj.:
  - `"tekst, "`



## Format JSON

Mało typów danych:

- string (ciąg znaków)
- liczby
- wartości prawda, fałsz
- pusta wartość typu `null`

Zwróćmy uwagę, że w tych typach danych nie ma np. daty ani czasu:

- Jeśli chcielibyśmy zawrzeć jakąś informację o dacie musielibyśmy ją przekazać w formie stringa bądź liczby.
- Wówczas nadawca i odbiorca powinni porozumieć się, w jaki sposób taką datę zakodować.

Typ numeryczny nie jest opisany przyjętymi standardami:

- liczba zmienna-przecinkowa na wejściu - zapisana w JSON, może mieć inną reprezentację w pamięci niż liczba zinterpretowana/wczytana.

JSON nie odróżnia atrybutów i wartości:

W XML'u możemy pobrać wartości na podstawie statusu np. zwróć tylko aktywnych użytkowników:

```
<person id="123" status="active">
  <name>Jan Kowalski</name>
  <age>30</age>
</person>
```

W JSON - wszystko to po prostu właściwości obiektu na tym samym poziomie:

```
{
  "person": {
    "id": "123",
    "status": "active",
```

JSON



```
    "name": "Jan Kowalski",  
    "age": 30  
  }  
}
```

Walidowanie JSON'a sprawdzonymi schematami (JSON Schema):

PYTHON

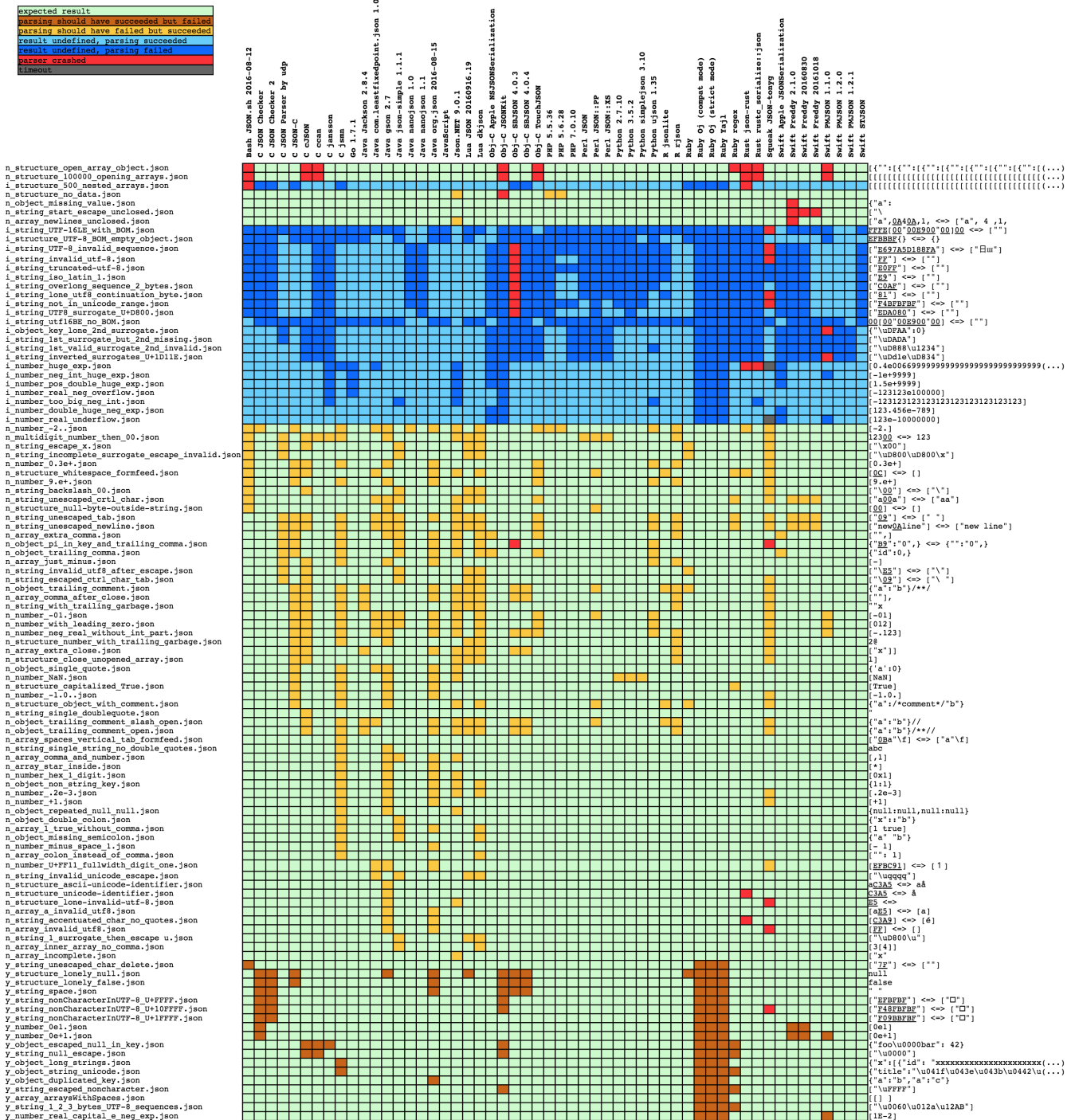
```
import json  
from jsonschema import validate  
  
config_schema = {  
    "type": "object",  
    "properties": {  
        "database": {  
            "type": "object",  
            "properties": {  
                "host": {"type": "string"},  
                "port": {"type": "integer", "minimum": 1, "maximum": 65535}  
            },  
            "required": ["host", "port"]  
        },  
    },  
    "required": ["database"]  
}  
  
def load_config(config_file):  
    with open(config_file) as f:  
        config = json.load(f)  
  
    validate(instance=config, schema=config_schema)  
    return config
```

Różne parsery JSON mają swoją własną interpretację wyjątków, które nie są ujęte w gramatyce JSON:

## JSON Parsing Tests, Pruned

Appendix to: [seriot.ch Parsing JSON is a Minefield](http://www.seriot.ch/parsing_json.php) [http://www.seriot.ch/parsing\\_json.php](http://www.seriot.ch/parsing_json.php)

2016-11-05 00:04:08



Dlatego często, pomimo tego, że poprawnie zamieniliśmy obiekt Python na JSON, możemy otrzymać błąd odpowiedzi z serwera, który inaczej zinterpretował ciało naszego zapytania HTTP.

## Format XML

- Rozbudowana składnia przypominająca strukturę HTML
- JSON - prosta składnia przypominająca swoją strukturą obiekt JavaScript

Skryptowaniem Excelem i PDF'em nie będziemy się zajmowa

Jak formatować datę?

- ISO 8601 tj. YYYY-MM-DD - większość formatów powinna poprawnie zinterpretować datę w powyższym formacie.

## GraphQL vs REST API:

---

- w GraphQL możemy w zapytaniu HTTP zdefiniować strukturę drzewiastą, jaką chcemy dostać w odpowiedzi.

## Narzędzia do pobierania danych z internetu:

---

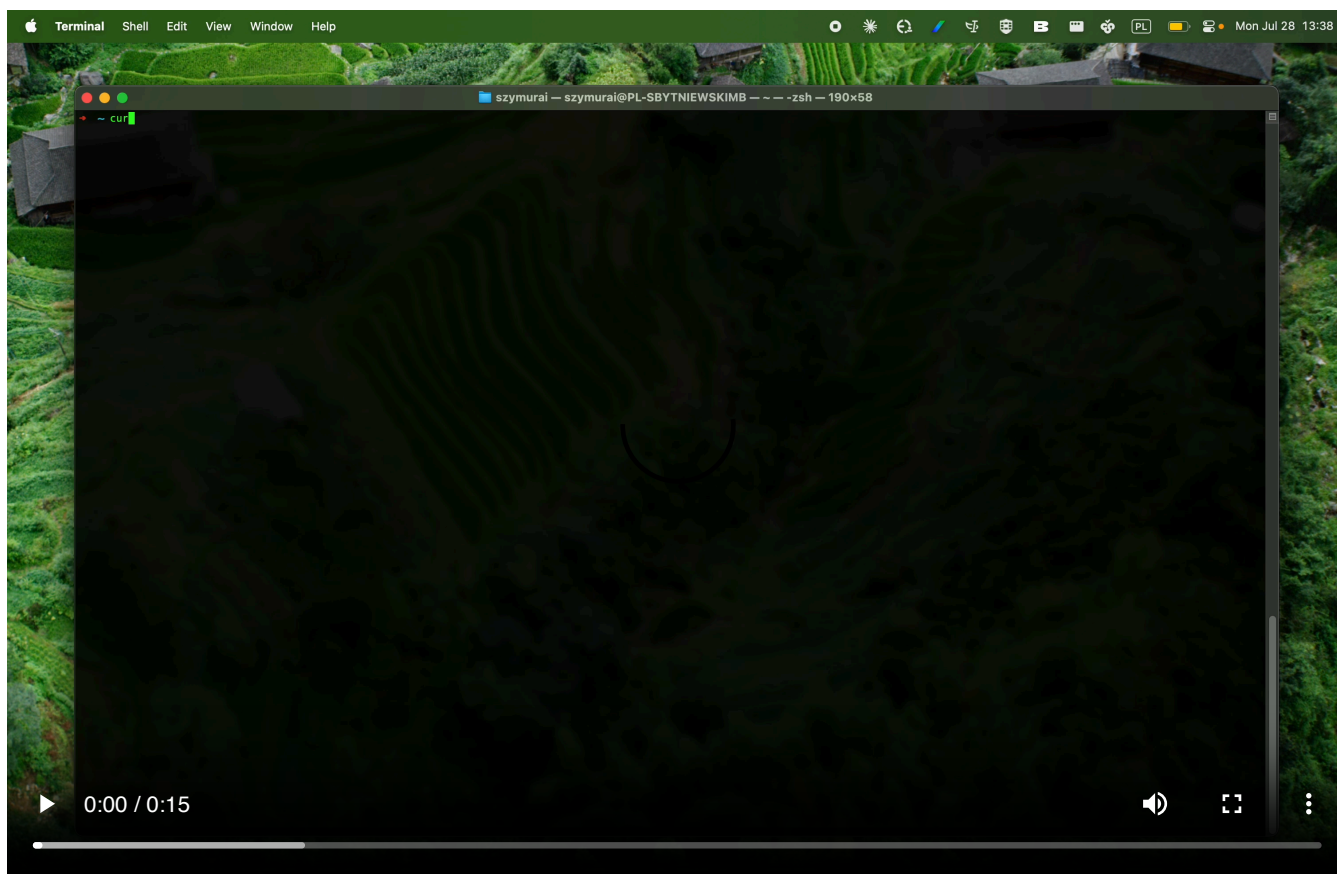
- curl - do prawie wszystkiego
- wget - możemy podać głębokość po jakiej ma schodzić po linkach (opcja rekursywnego pobierania całej strony)
- Każdy język programowania ma jakiś odpowiednik klienta HTTP:
  - requests - pakiet do osadzenia funkcjonalności curl we własnym programie Python.
  - Funkcjonalność sieciowa:
    - otworzy połączenie
    - wyśle żądanie
    - odczyta odpowiedź
    - status odpowiedzi
  - Wbudowane biblioteki mają swoje ograniczenia
- Dlatego możemy wcielić do swojego programu bibliotekę `libcurl` :
  - `pycurl` - wszystko to samo co możemy wykonać za pomocą narzędzia terminalowego `curl`
    - w sposób całkowicie świadomy możemy decydować co dzieje się np. z przekierowaniami (odpowiedzi o statusie 301)

## cURL

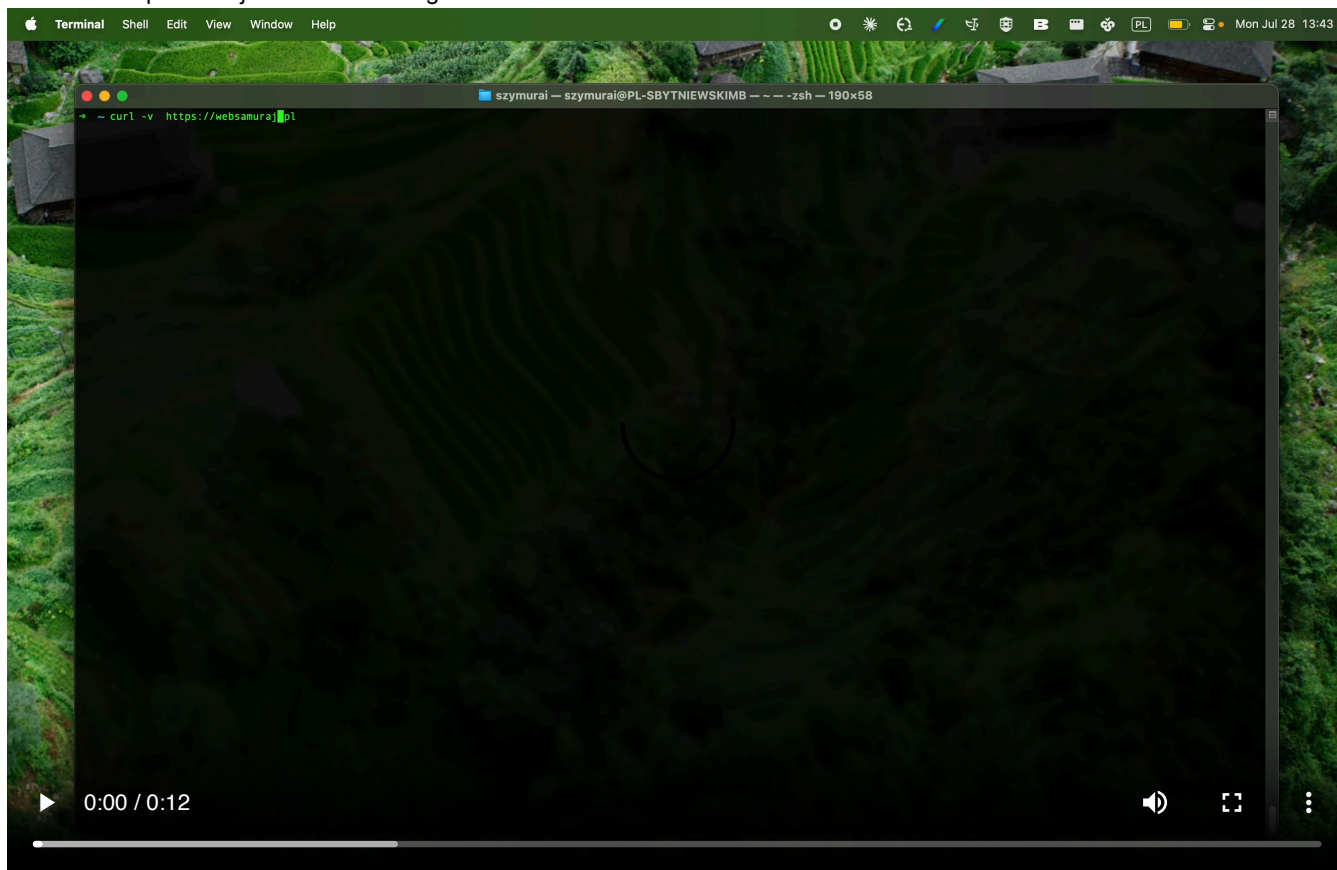
---

```
curl https://websamuraj.pl
```

SHELL



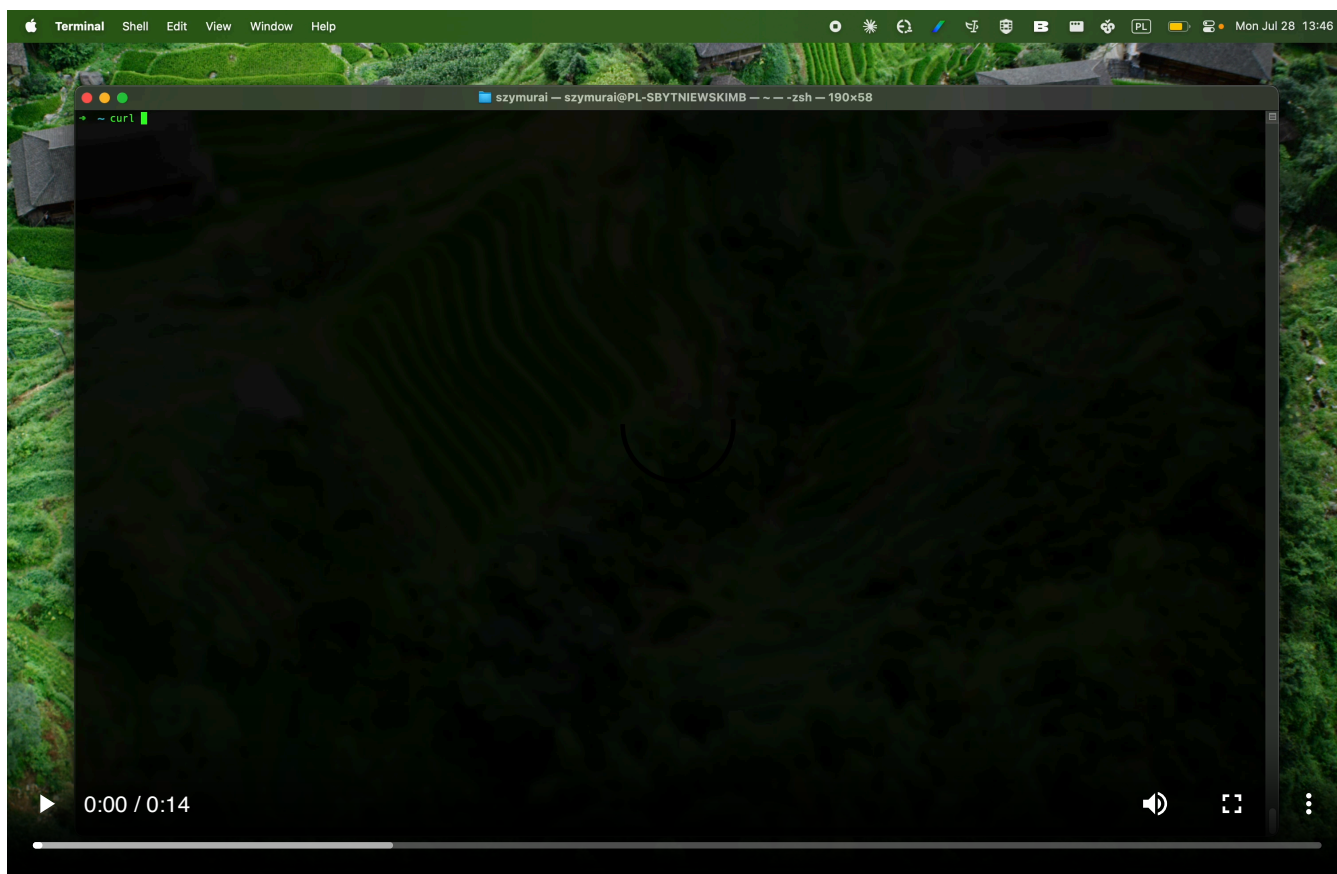
Jeśli jesteśmy zainteresowani, jakie były ustawione nagłówki żądania i odpowiedzi, możemy wybrać specjalną flagę `-v` (verbose), która oznacza "gadatliwość" odpowiedzi. Dodatkowo wpisując flagę `-I` dostaniemy odpowiedź bez HTML'a. Jak widać odpowiedź jest bardzo szczegółowa:



Gdy pobieramy stronę, możemy ją bezpośrednio przekierować do pliku:

```
curl https://websamuraj.pl > strona_www
```





Zawartość strony www zostanie zapisana do domyślnego katalogu:



Przydatne opcje

-I

- wyświetlanie tylko nagłówków  
`-o bądź -O`
- zapis pobranych danych do pliku  
`-H "key: value"`
- wstawienie dowolnego nagłówka do zapytania HTTP:
  - czasami serwisy wymagają następującego nagłówka (zwłaszcza API):
    - "Content-Type: application/json"`--limit-rate 200K`
- ograniczenie prędkości pobierania  
`--insecure`
- ignorowanie błędnych certyfikatów  
`-F filename=image_name.jpg -F upload=@image_name.jpg`
- wysyłanie pliku w żądaniu `POST`

»

## Narzędzia działające na wyższym poziomie abstrakcji:

- bardziej niezawodnie
- szybciej

## Scrapy

- narzędzie które automatycznie chodzi po jakiejś serii stron tj. wykorzystywane do pobierania danych z zasobów, które są stronicowane.

Ten kod to spider (pająk) napisany w frameworku Scrapy do scrapowania (pobierania danych) ze strony internetowej z cytatami.

```
import scrapy

class QuotesSpider(scrapy.Spider):
    name = 'quotes'
    start_urls = [
        'http://quotes.toscrape.com/tag/humor/',
    ]

    def parse(self, response):
        # Iteruje przez wszystkie cytaty na stronie
        for quote in response.css('div.quote'):
            yield {
                'author': quote.xpath('span/small/text()').get(),
                'text': quote.css('span.text::text').get(),
            }

        # Znajdź link do następnej strony
        next_page = response.css('li.next a::attr("href")').get()
        if next_page is not None:
            # Przejdź do następnej strony i wywołaj ponownie metodę parse
            yield response.follow(next_page, self.parse)
```

PYTHON

## Playwright

---

Naśladowanie zachowania użytkowników wykorzystując driversy tj. specjalnie zaprojektowaną przeglądarkę uruchamianą z poziomu skryptu Python.

- problemy ze skalowaniem takich aplikacji
- bardzo czasochłonne wykonywanie akcji.



## Requests

---

Biblioteka Python obsługująca funkcjonalności klienta HTTP.

```
from requests import get
```

- pobiera zawartość strony - podobnie jak przeglądarka.

```
https://danepubliczne.imgw.pl/api/data/synop
```

Dane względem danej stacji:

```
https://danepubliczne.imgw.pl/api/data/synop/station/gdansk
```

Ostrzeżenia meteorologiczne:

```
https://danepubliczne.imgw.pl/api/data/warningsmeteo
```

<https://www.wordsapi.com>

## Bonus:

---

<https://api.together.ai>